Assignment Solution: Software Engineering & DevOps Principles

Module: Version Control & Test Automation

Student Name: Shahriar Rahman

GitHub Repository: https://github.com/rahmanshahriar777/myuniversityproject.git

Submission Format: Single PDF Document

Total Marks: 20 / 20 (Q1: 10 Marks, Q2: 10 Marks)

Question 1: Git & GitHub Workflow (10 Marks)

This section demonstrates a complete, professional Git version control workflow executed entirely on the local machine and synchronized with a public GitHub repository. It covers repository initialization, multi-stage commits, branching, merging, exclusion policies via `.gitignore`, commit rollback operations, and public remote deployment.

Step 1: Local Repository Setup 2 Marks

A project directory was created on the local filesystem without pre-cloning or initializing on GitHub first. The baseline project files (`index.html` and `about.html`) were authored locally, and Git was initialized.

```
# 1. Author initial HTML files
New-Item index.html, about.html

# 2. Initialize local Git repository and set primary branch to main
git init
Initialized empty Git repository in C:/Users/user/OneDrive/Desktop/MY PROJEC/.git/

git branch -M main
git config user.name "Shahriar Rahman"
git config user.email "rahmanshahriar777@gmail.com"

git status
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        about.html
        index.html
```

Step 2: Git Workflow Execution 4 Marks

The core workflow was executed in sequential order to establish a clean, traceable commit history showcasing isolated branching and non-fast-forward merge activity:

1. **Initial Commit:** Staged and committed baseline files `index.html` and `about.html`.
2. **Second Commit:** Enhanced `about.html` with course and assignment metadata.

3. **Branch Creation & Switch:** Created and switched to isolated branch `feature`.
4. **Feature Commit:** Added interactive feature cards for assignments and grades in `index.html`.
5. **Branch Switch:** Returned to `main` branch.
6. **Merge:** Merged `feature` into `main` using `--no-ff` to record explicit merge topology.

```
# 1. Initial Commit
git add index.html about.html
git commit -m "Initial commit: Add index.html and about.html"
[main (root-commit) 11a43c8] Initial commit: Add index.html and about.html
 2 files changed, 64 insertions(+)
 create mode 100644 about.html
 create mode 100644 index.html

# 2. Modify about.html and create second commit
git add about.html
git commit -m "Update about.html with course details and objectives"
[main 2af4ff9] Update about.html with course details and objectives
 1 file changed, 9 insertions(+)

# 3. Create and switch to 'feature' branch
git branch feature
git checkout feature
Switched to branch 'feature'

# 4. Modify index.html on feature branch and commit
git add index.html
git commit -m "Add feature cards for assignments and grades on feature branch"
[feature b35f2d2] Add feature cards for assignments and grades on feature branch
 1 file changed, 11 insertions(+)

# 5. Switch back to main branch
git checkout main
Switched to branch 'main'

# 6. Merge feature branch into main with explicit merge commit
git merge --no-ff feature -m "Merge feature branch into main"
Merge made by the 'ort' strategy.
 index.html | 11 ++++++++++
 1 file changed, 11 insertions(+)
```

Commit History & Branch Graph Output:

```
git log --graph --oneline --decorate --all
*   7de0e55 (HEAD -> main) Merge feature branch into main
|\
| * b35f2d2 (feature) Add feature cards for assignments and grades on feature branch
|/
* 2af4ff9 Update about.html with course details and objectives
* 11a43c8 Initial commit: Add index.html and about.html
```

Step 3: .gitignore Implementation 1 Mark

A `.gitignore` file was constructed in the root directory to automatically exclude build artifacts, sensitive logs, and temporary directories from version control tracking.

> **Contents of .gitignore:**
> `*.log` — Matches and ignores all log files in any directory.
> `temp/` — Matches and excludes the temporary folder and all nested files.
> `.kilo/` — Excludes local agent and IDE metadata.

```
# Verification of .gitignore functionality
New-Item -ItemType Directory -Force -Path temp
"Error log dump" | Out-File -FilePath debug.log
"Cached runtime data" | Out-File -FilePath temp\cache.tmp

git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (debug.log and temp/ are successfully
ignored!)

git check-ignore -v debug.log temp\cache.tmp
.gitignore:2:*.log      debug.log
.gitignore:5:temp/      temp\cache.tmp

git add .gitignore
git commit -m "Add .gitignore to exclude log files, temp folder, and editor metadata"
[main 7a3688b] Add .gitignore to exclude log files, temp folder, and editor metadata
 1 file changed, 8 insertions(+)
 create mode 100644 .gitignore
```

Step 4: Undo Operation using Git 1 Mark

To demonstrate Git's rollback capabilities, an unwanted commit was introduced and subsequently undone using `git reset HEAD~1`, demonstrating how the repository pointer moves backward while preserving working directory control.

```
# 1. Introduce an unwanted commit
"Temporary experimental notes" | Out-File -FilePath experimental_feature.txt
git add experimental_feature.txt
git commit -m "Add experimental feature draft (to be undone)"
[main fb92e56] Add experimental feature draft (to be undone)
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 experimental_feature.txt

# 2. View commit history before undo
git log --oneline -n 2
fb92e56 Add experimental feature draft (to be undone)
7a3688b Add .gitignore to exclude log files, temp folder, and editor metadata

# 3. Perform undo operation (mixed reset moves HEAD back 1 commit)
git reset HEAD~1
Unstaged changes after reset:
Untracked files:
        experimental_feature.txt

# 4. Verify repository state after undo
git log --oneline -n 2
7a3688b Add .gitignore to exclude log files, temp folder, and editor metadata
7de0e55 Merge feature branch into main

git status
On branch main
Untracked files:
        experimental_feature.txt
(Commit successfully undone; HEAD reset back to 7a3688b without data loss)
```

> **Operation Assessment:** `git reset HEAD~1` safely dismantled the commit object `fb92e56` from the branch timeline. For public/collaborative branches, `git revert HEAD` can alternatively be used to record an inverse commit without rewriting history.

Step 5: GitHub Integration 1 Mark

The local repository was connected to a public repository on GitHub and pushed to establish remote availability for both `main` and `feature` branches.

```
git remote add origin https://github.com/rahmanshahriar777/myuniversityproject.git
git remote -v
origin  https://github.com/rahmanshahriar777/myuniversityproject.git (fetch)
origin  https://github.com/rahmanshahriar777/myuniversityproject.git (push)

# Push main branch with upstream tracking
git push -u origin main
To https://github.com/rahmanshahriar777/myuniversityproject.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

# Push feature branch for full history audit
git push origin feature
To https://github.com/rahmanshahriar777/myuniversityproject.git
 * [new branch]      feature -> feature
```

Public GitHub Repository URL:
https://github.com/rahmanshahriar777/myuniversityproject.git

Step 6: Command Documentation File (git_commands.txt) 1 Mark

The file `git_commands.txt` was generated in the project root, committed to the repository, and pushed to GitHub. It documents every command utilized during the assignment alongside 1–2 line concise technical explanations.

Question 2: Test Case Design (10 Marks)

Application Scenario: Testing an enterprise Authentication / Login System with three core user interface components: **Username** (text input), **Password** (masked text input), and **Login Button** (submission trigger).

> **Functional Requirements (FR) Specification:**
>
> - **FR-01:** Username must not be empty.
> - **FR-02:** Password must be at least 6 characters long.
> - **FR-03:** Valid username and password allow a successful login.
> - **FR-04:** Invalid credentials should display an appropriate error message.
> - **FR-05:** After 3 consecutive failed login attempts, the account should be locked.

Test Design Techniques Applied:

- **Equivalence Partitioning (EP):** Partitioning inputs into valid partitions (non-empty username, password ≥ 6 characters, registered account) and invalid partitions (empty fields, password < 6 characters, unregistered credentials).
- **Boundary Value Analysis (BVA):** Evaluating lengths around the 6-character threshold (length = 0, length = 5, length = 6, length = 7).
- **State Transition Testing:** Verifying system behavior across failed attempts (Attempt 1 → Attempt 2 → Attempt 3 [Locked] → Attempt on Locked Account → Successful Reset).
- **Security & Error Guessing:** Preventing username enumeration by verifying generic messages for unregistered accounts and testing whitespace-only inputs.

Comprehensive Test Case Table

ID	Test Scenario	Input Data	Expected Output	Coverage
`TC_01`	**Valid Login (Standard)** Authenticate registered user with strong password exceeding 6 chars. **Positive / Valid**	**Username:** `shahriar_user` **Password:** `SecurePass123!` (14 chars) **Action:** Click "Login"	Authentication succeeds. System creates an active user session, generates security JWT token, and redirects to `/dashboard.html` displaying "Welcome back, Shahriar Rahman".	FR-03
`TC_02`	**Boundary Value Analysis (Minimum Valid Password)** Authenticate registered user with password exactly 6 characters long. **Edge Case / BVA**	**Username:** `john_doe` **Password:** `Pass12` (Exactly 6 chars) **Action:** Click "Login"	Password length validation passes. User is authenticated successfully and redirected to `/dashboard.html`. No validation error triggered.	FR-02 FR-03

ID	Test Scenario	Input Data	Expected Output	Coverage
TC_03	**Empty Username Field** Attempt login with an empty username and a valid password. **Negative / Validation**	**Username:** "" (Empty string) **Password:** ValidPass123 **Action:** Click "Login"	Submission is blocked. Field-level validation error appears below username input: "*Username is required and cannot be empty.*" Cursor focuses on username field. No network request sent to auth API.	FR-01
TC_04	**Empty Password Field** Attempt login with a valid username and empty password. **Negative / Validation**	**Username:** shahriar_user **Password:** "" (Empty string) **Action:** Click "Login"	Submission is blocked. Field-level validation error appears below password input: "*Password is required and cannot be empty.*" Cursor focuses on password field. No network request sent to auth API.	FR-01 FR-02
TC_05	**Both Fields Empty** Attempt login without entering username or password. **Negative / Validation**	**Username:** "" **Password:** "" **Action:** Click "Login"	Submission is blocked. Dual validation messages appear: "*Username is required*" and "*Password is required*". Form submission aborted.	FR-01 FR-02
TC_06	**Whitespace-Only Username** Attempt login with whitespace spaces in username. **Edge Case**	**Username:** " " (3 whitespace spaces) **Password:** ValidPass123 **Action:** Click "Login"	System trims input, identifies empty string post-trim, and displays error: "*Username cannot consist solely of whitespace spaces.*" Form is not submitted.	FR-01
TC_07	**Boundary Value Analysis (Password < 6 Characters)** Attempt login with 5-character password (one below minimum). **Edge Case / BVA**	**Username:** shahriar_user **Password:** abc12 (5 chars) **Action:** Click "Login"	Submission blocked by client/server validation. Displays error message: "*Password must be at least 6 characters long.*" Authentication is not executed.	FR-02
TC_08	**Invalid Credentials (Attempt 1 of 3)** Valid username with incorrect password on active account. **Invalid Scenario**	**Username:** shahriar_user **Password:** WrongPass01 **Action:** Click "Login"	Authentication fails. System increments failed login counter to 1. Displays error banner: "*Invalid username or password. 2 attempts remaining before account lockout.*" Account remains ACTIVE.	FR-04 FR-05
TC_09	**Invalid Credentials (Attempt 2 of 3)** Second consecutive failed login attempt on same account. **Invalid Scenario**	**Username:** shahriar_user **Password:** WrongPass02 **Action:** Click "Login"	Authentication fails. Failed login counter increments to 2. Warning banner displayed: "*Invalid username or password. Warning: 1 attempt remaining before account lockout.*" Account remains ACTIVE.	FR-04 FR-05

ID	Test Scenario	Input Data	Expected Output	Coverage
TC_10	**Account Lockout Trigger (Attempt 3 of 3)** Third consecutive failed login attempt. Security / Lockout	**Username:** shahriar_user **Password:** WrongPass03 **Action:** Click "Login"	Authentication fails. Failed attempt counter reaches 3. Account status updates to LOCKED in database. Alert displayed: "*Your account has been locked due to 3 consecutive failed login attempts. Please reset password or contact support.*"	FR-04 FR-05
TC_11	**Login Attempt on Locked Account** Attempt login on locked account with correct credentials. Security / Lockout	**Username:** shahriar_user (Status: LOCKED) **Password:** SecurePass123! (Correct password) **Action:** Click "Login"	Authentication rejected despite correct password. System displays modal message: "*Account is currently locked. Please check your email for unlock instructions or contact an administrator.*" Access denied.	FR-05
TC_12	**Failed Counter Reset on Successful Login** User has 2 prior failed attempts, then enters correct password. State Transition	**Precondition:** Account failed_attempts = 2 **Username:** shahriar_user **Password:** SecurePass123! **Action:** Click "Login"	Authentication succeeds. System resets the failed login attempt counter from 2 back to 0. Session token created and user redirected to Dashboard.	FR-03 FR-05
TC_13	**Non-Existent / Unregistered User** Attempt login with username that does not exist in database. Negative / Security	**Username:** non_existent_student_99 **Password:** SomePass123 (Valid format) **Action:** Click "Login"	Authentication fails. Displays generic error banner: "*Invalid username or password.*" (System prevents user enumeration by not revealing whether username exists).	FR-04

Requirements Traceability Matrix (RTM)

Requirement ID	Requirement Description	Associated Test Cases	Status
FR-01	Username must not be empty	TC_03, TC_05, TC_06	**Covered (100%)**
FR-02	Password must be at least 6 characters long	TC_02, TC_04, TC_05, TC_07	**Covered (100%)**
FR-03	Valid username and password allow a successful login	TC_01, TC_02, TC_12	**Covered (100%)**
FR-04	Invalid credentials should display an error message	TC_08, TC_09, TC_10, TC_13	**Covered (100%)**
FR-05	After 3 consecutive failed login attempts, account is locked	TC_08, TC_09, TC_10, TC_11, TC_12	**Covered (100%)**

End of Assignment Document • Submitted by Shahriar Rahman • Verified Git Repository: https://github.com/rahmanshahriar777/myuniversityproject.git